applied sciences

MDPI

Article

CurriculumPT: LLM-Based Multi-Agent Autonomous Penetration Testing with Curriculum-Guided Task Scheduling

Xingyu Wu [1,2], Yunzhe Tian [1,2], Yuanwan Chen [1,2], Ping Ye [1,2], Xiaoshu Cui [1,3], Jingqi Jia [1,2], Shouyang Li [4], Jiqiang Liu [1,2] and Wenjia Niu [1,2,*]

[1] Beijing Key Laboratory of Security and Privacy in Intelligent Transportation, Beijing Jiaotong University, Beijing 100044, China; xingyu_wu@bjtu.edu.cn (X.W.); tianyunzhe@bjtu.edu.cn (Y.T.); chenyuanwan@bjtu.edu.cn (Y.C.); 23120489@bjtu.edu.cn (P.Y.); cuixiaoshu@bjtu.edu.cn (X.C.); jingqijia@bjtu.edu.cn (J.J.); jqliu@bjtu.edu.cn (J.L.)
[2] School of Cyberspace Science and Techonology, Beijing Jiaotong University, Beijing 100044, China
[3] School of Computer Science and Technology, Beijing Jiaotong University, Beijing 100044, China
[4] Construction Department, China Railway Qinghai Tibet Group Co., Ltd., Xining 810000, China; xxm450360@163.com
* Correspondence: niuwj@bjtu.edu.cn

Abstract

While autonomous driving systems and intelligent transportation infrastructures become increasingly software-defined and network-connected, ensuring their cybersecurity has become a critical component of traffic safety. Large language models (LLMs) have recently shown promise in automating aspects of penetration testing, yet most existing approaches remain limited to simple, single-step exploits. They struggle to handle complex, multi-stage vulnerabilities that demand precise coordination, contextual reasoning, and knowledge reuse. This is particularly problematic in safety-critical domains, such as autonomous vehicles, where subtle software flaws can cascade across interdependent subsystems. In this work, we present CurriculumPT, a novel LLM-based penetration testing framework specifically designed for the security of intelligent systems. CurriculumPT combines curriculum learning and a multi-agent system to enable LLM agents to progressively acquire and apply exploitation skills across common vulnerabilities and exposures-based tasks. Through a structured progression from simple to complex vulnerabilities, agents build and refine an experience knowledge base that supports generalization to new attack surfaces without requiring model fine-tuning. We evaluate CurriculumPT on 15 real-world vulnerabilities scenarios and demonstrate that it outperforms three state-of-the-art baselines by up to 18 percentage points in exploit success rate, while achieving superior efficiency in execution time and resource usage. Our results confirm that CurriculumPT is capable of autonomous, scalable penetration testing and knowledge transfer, laying the groundwork for intelligent security auditing of modern autonomous driving systems and other cyberphysical transportation platforms.

Keywords: autonomous driving security; intelligent transportation systems; penetration testing; large language models; multi-agent system; curriculum learning; sensor-based cyberphysical systems

Academic Editor: Rui Araújo

Received: 9 July 2025
Revised: 7 August 2025
Accepted: 13 August 2025
Published: 18 August 2025

Citation: Wu, X.; Tian, Y.; Chen, Y.; Ye, P.; Cui, X.; Jia, J.; Li, S.; Liu, J.; Niu, W. CurriculumPT: LLM-Based Multi-Agent Autonomous Penetration Testing with Curriculum-Guided Task Scheduling. *Appl. Sci.* **2025**, *15*, 9096. https://doi.org/10.3390/app15169096

1. Introduction

With the proliferation of intelligent transportation systems and the rapid deployment of autonomous driving, ensuring cybersecurity has become a foundational pillar for traffic

safety and road system resilience. Modern autonomous driving platforms integrate complex cyberphysical architectures that include real-time operating systems, networked electronic control units (ECUs), and vehicular communication modules (e.g., V2X), as well as diverse sensor suites such as LiDAR, GPS, and cameras. These components collectively expose a broad and dynamic attack surface. In this context, penetration testing [1,2] serves as a proactive mechanism to assess the security posture of such intelligent systems. However, traditional penetration testing [3] remains heavily reliant on human expertise, rendering it impractical for large-scale, continuous, and adaptive evaluation in safety-critical domains like autonomous driving. This limitation has intensified the demand for automated, intelligent penetration testing solutions [4] capable of operating across complex, multi-stage attack chains and identifying novel threats, including zero-day vulnerabilities [5,6].

Recent efforts have explored diverse automation strategies to advance penetration testing beyond traditional, manual paradigms. Tudosi et al. [7] performed comprehensive assessments on distributed firewall deployments and revealed that even widely adopted solutions such as pfSense exhibit exploitable security flaws under targeted probing. Their findings highlight the critical need for combining automated scanning with human-in-the-loop analysis to uncover subtle, system-wide vulnerabilities. In parallel, Chowdhary et al. [8] introduced a GAN-based framework that autonomously generates adversarial web attack payloads capable of bypassing modern Web Application Firewalls (WAFs). Their approach demonstrates the efficacy of generative models in crafting evasive, realistic attack vectors—particularly in application-layer contexts such as cross-site scripting (XSS) and SQL injection. In a related but orthogonal direction, Berenguer et al. [9] investigated the use of large language models (LLMs) to extract and transform raw sensor data from unstructured formats (e.g., HTML) into structured representations (e.g., JSON, XML). While primarily targeting data interoperability, their work showcases the broader potential of LLMs for robust parsing, semantic interpretation, and transformation of machine-generated outputs—capabilities that are increasingly relevant for automated vulnerability analysis and exploit generation.

These developments collectively signal a broader trend: leveraging AI-driven models, particularly LLMs and generative frameworks, to replace or augment traditional manual security workflows. With strong capabilities in language understanding, reasoning, code generation and tool interaction [10], LLMs provide a powerful alternative to conventional AI planning or reinforcement learning (RL) agents, which often struggle with generalization and scalability. Early explorations in this direction include the work of Happe et al. [11], which constructed a closed-loop system using GPT-3.5 to autonomously interact with vulnerable virtual machines. Despite its feasibility, it was limited to CTF-style tasks and lacked modularity or learning mechanisms. PentestGPT [12] is one of the earliest attempts to apply LLMs to real-world penetration testing. It leverages an Auto-GPT [13] architecture to plan and guide attack steps but requires human intervention for command execution, limiting its autonomy. However, these early systems treat each attack task in isolation and lack a mechanism to accumulate and transfer knowledge across tasks. Without the ability to build up reusable experience, their generalization and long-term planning capabilities remain limited.

Building on early explorations, recent research has shifted toward the multi-agent system (MAS) based on LLMs to better emulate the division of labor observed in real-world penetration testing teams. This design enables role specialization, improves task modularity, and facilitates coordinated execution across complex attack phases. For instance, AutoAttacker [14] proposes a modular MAS architecture in which LLM agents collaborate on post-exploitation tasks, supported by a retrieval-augmented generation (RAG)-based experience manager for limited knowledge reuse. AutoPT [15] introduces a

Penetration testing State Machine (PSM), using finite-state control to guide LLM agents through web penetration stages, helping reduce planning instability in long-horizon tasks. Similarly, VulnBot [16] constructs a full-stack MAS framework where LLM agents handle reconnaissance, vulnerability analysis, and exploitation, coordinated through a shared Penetration Task Graph (PTG). Extending these MAS paradigms to the domain of intelligent transportation, Gao et al. [17] explored the use of LLMs in simulating traffic system behaviors and generating cybersecurity strategies in autonomous driving scenarios. Their work highlights the feasibility of leveraging LLMs not only for modeling vehicular communication and sensor interactions, but also for identifying attack vectors and evaluating potential impacts on traffic safety. While these systems demonstrate notable improvements in autonomy and success rates compared to single-agent baselines, they still suffer from a fundamental limitation: the lack of structured, progressive skill acquisition. In all cases, agents are exposed directly to high-difficulty, multi-stage common vulnerabilities and exposures (CVE) exploitation tasks without prior scaffolding or experiential buildup, leading to low success rates (e.g., VulnBot reports only 20% on complex vulnerabilities), weak generalization, and fragile reasoning when handling logical flaws or chained exploits. Critically, none of these methods reflect the human-like process of accumulating expertise over time through a curriculum of increasing difficulty. This absence of structured experiential learning has emerged as a key bottleneck limiting the scalability and adaptability of current LLM-based pentesting systems.

These limitations are particularly problematic in safety-critical domains like autonomous driving, where vulnerabilities may cascade across multiple components—from perception to control—and compromise not only system integrity but also physical safety. To bridge this critical gap, we propose CurriculumPT, an automated penetration testing framework designed to enable agents to learn systematically. Our approach uniquely integrates three core components: curriculum learning (CL), a multi-agent system (MAS), and an experience knowledge base (EKB). To solve the problem of unstructured skill acquisition, the CL module organizes CVEs into a curriculum of increasing difficulty, allowing agents to build expertise incrementally without costly fine-tuning, thereby mimicking human learning patterns. This curriculum guides our MAS module, where specialized LLM agents (e.g., planner, recon, exploiter) collaborate to execute tasks. Their strategies and focus adapt based on the curriculum's difficulty stage. Finally, to ensure knowledge retention and transfer, the EKB module serves as a central memory. It captures and organizes successful strategies, toolchains, and decision rationales from completed tasks. This dynamic knowledge base is then retrieved by agents to tackle more complex challenges, fostering generalization. By tightly coupling these components, CurriculumPT creates a virtuous cycle of learning and application, enabling agents to autonomously accumulate experience and achieve superior performance on real-world penetration tests.

We systematically evaluate CurriculumPT against three state-of-the-art LLM-based penetration testing systems (AutoPT [15], VulnBot [16], and PentestAgent [18]) on 15 real-world CVE exploitation scenarios. The results unequivocally demonstrate the superiority of our learning-centric approach. CurriculumPT achieves the highest average exploit success rate (ESR), outperforming the strongest baseline by a significant 18 percentage points. This success is complemented by superior efficiency: CurriculumPT reduces both average task execution time (by 20.6%) and token usage (by 25.5%), and requires the fewest decision-making steps. These efficiency gains underscore that our framework learns more direct and effective exploitation strategies, rather than relying on brute-force attempts. Furthermore, ablation studies confirm that the curriculum is essential for effective skill acquisition, while the knowledge base is critical for adaptability and knowledge reuse. Together, these findings validate that CurriculumPT is a more effective, efficient, and scalable solution for complex,

multi-stage autonomous penetration testing. The main contributions of this work can be summarized as follows:

- We pioneer the integration of curriculum learning into multi-agent LLM-based penetration testing. This novel approach systematically addresses the critical challenge of skill acquisition and generalization, enabling agents to progressively master complex tasks without human intervention or model fine-tuning.
- We design and implement a synergistic framework, CurriculumPT, that tightly couples a curriculum scheduler, specialized agents, and a dynamic experience knowledge base. This architecture creates a closed-loop system for continuous, autonomous learning and knowledge transfer in complex environments.
- We conduct extensive experiments on a benchmark of real-world CVE tasks, demonstrating that our method achieves a 15–36% improvement in exploit success rate (ESR) over representative baselines, along with significantly lower execution time and token usage, confirming its superior efficiency and effectiveness.

The rest of this paper is organized as follows. Section 2 reviews related work on LLM-based penetration testing and curriculum learning. Section 3 presents the overall architecture of the proposed CurriculumPT framework, including the curriculum construction strategy, task scheduling mechanism, multi-agent collaboration design, and the experience knowledge base. Section 4 describes the experimental setup, evaluation metrics, and benchmark scenarios, and reports results from comparative and ablation studies. Section 5 discusses the core contributions of our approach, challenges for real-world deployment, and its limitations. Finally, Section 6 concludes the work and outlines potential directions for future research.

2. Related Work

The subsequent literature review provides an overview of related work focusing on: the evolution of LLM-based automated penetration testing systems, and the application of curriculum learning as a strategy for structured skill acquisition and generalization.

2.1. Large Language Models in Penetration Testing

Penetration testing (pentesting) has long been a labor-intensive process that requires significant human expertise to uncover, analyze, and exploit vulnerabilities within complex systems. To reduce manual effort and scale testing across large infrastructures, researchers initially turned to AI planning and reinforcement learning (RL) to automate pentesting workflows. These methods often framed the task as a sequential decision problem, using attack graphs or state-action spaces to simulate optimal attack paths. While RL-based frameworks such as hierarchical RL [19,20] and imitation learning [21] showed promising results in simulated environments like NASim [22] or CybORG [23], they remained constrained by several critical limitations. Specifically, RL agents typically require environment-specific training, struggle to generalize across unseen targets, rely heavily on structured prior knowledge (e.g., network topologies), and falter in handling complex, multi-stage exploits. These shortcomings limit their applicability in real-world penetration scenarios where adaptability, reasoning, and knowledge reuse are essential.

In light of these limitations, researchers have increasingly turned to LLMs to bring more flexible reasoning and contextual understanding to automated penetration testing. Early explorations, such as PentestGPT [12], used GPT-3.5/4 to assist human testers by generating attack commands and step-by-step guidance [24–26]. Although these systems demonstrated the potential of LLMs in security workflows, they still relied on human intervention for tool execution and decision making. To achieve greater autonomy, more recent work has integrated LLMs directly into the attack loop. These frameworks like AutoAttacker [14],

HackSynth [27], and PentestAgent [18] demonstrated that LLMs could autonomously conduct reconnaissance, generate payloads, exploit vulnerabilities, and adapt based on tool feedback. Despite this progress, these single-agent systems still face challenges in scaling to complex, multi-phase attack scenarios. Their generalization capabilities remain limited, especially when tackling novel vulnerabilities or chained exploits that require memory, collaboration, or multi-step reasoning.

In response to these limitations, another emerging direction has adopted the multi-agent system (MAS) as a extension to LLM-based pentesting. Drawing inspiration from human red teams, where individuals specialize in tasks like scanning, privilege escalation, or post-exploitation, MAS-based approaches distribute tasks among multiple LLM agents with dedicated roles. VulnBot [16] exemplifies this strategy by orchestrating a team of agents that share knowledge through a penetration task graph, collaboratively tackling reconnaissance, scanning, and exploitation phases. Empirical results show that such division of labor improves both exploit success rates and system robustness compared to monolithic LLM agents. Other works such as Pentest-AI [28], D-CIPHER [29] and RapidPen [30] follow a similar trend, demonstrating that role-specific agent coordination mitigates reasoning bottlenecks and enhances task coverage. Nevertheless, these frameworks still lack a mechanism for systematic skill accumulation and experiential learning, which limits their ability to adapt to more difficult, previously unseen attack paths.

To overcome these learning limitations, we introduce CurriculumPT. Our framework integrates curriculum-driven learning process directly into multi-agent penetration testing system. By organizing tasks in a structured, easy-to-hard progression and enabling knowledge reuse through a dynamic experience base, CurriculumPT treats pentesting not as a series of isolated problems, but as a continuous learning process. This approach, detailed further in the context of curriculum learning below, directly addresses the gaps in skill acquisition and generalization left by prior work.

2.2. Curriculum Learning

Curriculum learning [31] (CL) is a machine learning training strategy inspired by the human learning process of progressing from easy to difficult tasks. It involves initially presenting the model with simple samples or tasks, then gradually increasing the difficulty until the model can handle complex target tasks. This progressive approach is believed to help models converge faster and achieve better final performance and generalization. The core of CL lies in determining how to reasonably define task difficulty and design an effective curriculum sequence. CL has achieved success in various machine learning applications; for example, in software vulnerability detection, Du et al. [32] introduced CL to arrange training samples from easy to difficult to learn better detection models. In reinforcement finetuning (RFT) of LLMs, Zhang et al. [33] proposed ADARFT, which dynamically adjusts the difficulty of training problems through adaptive curriculum learning to improve the efficiency and accuracy of RFT. Furthermore, the idea of curriculum learning has been applied to improve in-context learning (ICL) for LLMs. For instance, the CDS method proposed by Vu et al. [34] partitions samples by their complexity and selects demonstrations from easy to difficult to cover a wide range of difficulty levels, thereby enhancing LLM learning. Ma et al. [35] also proposed a curriculum ICL strategy guided by problem-solving logic, selecting and ordering demonstration examples by analyzing this logic. Similarly, the ADCL strategy by Zhang et al. [36] addresses the "Difficulty Shift" phenomenon by periodically re-estimating difficulty within upcoming data batches to maintain alignment with the model's evolving capabilities.

However, in the context of LLM-driven automated penetration testing, curriculum learning remains largely unexplored. Existing systems such as AutoAttacker [14] and

VulnBot [16] incorporate memory modules or retrieval-based augmentation to reuse prior knowledge, but they lack an explicit progression of task difficulty to guide systematic skill acquisition. As a result, LLM agents are often exposed directly to complex, multi-stage CVE scenarios without sufficient grounding, leading to low exploitation success rates and poor generalization. In this work, we make the first attempt to integrate curriculum learning into LLM-based autonomous penetration testing. Our CurriculumPT framework introduces a structured task scheduling mechanism that organizes real-world CVE exploitation tasks into a difficulty-graded curriculum. This progression enables a multi-agent LLM system to accumulate actionable experience in a staged manner, learning reusable strategies and decision patterns from simpler tasks and transferring them to tackle more complex vulnerabilities. Crucially, our approach does not require model fine-tuning or external supervision. Instead, the curriculum is used to shape the agents' experiential learning trajectory and improve their reasoning and adaptability. This explicit coupling of curriculum learning with multi-agent LLM orchestration constitutes a key innovation that addresses core limitations in scalability, skill transfer, and generalization found in prior work.

3. Methodology

In this section, we first present an overview of the CurriculumPT architecture (Section 3.1). We then detail its core learning mechanisms, including curriculum design and experience management (Section 3.2). Next, we describe the curriculum-guided multi-agent system that acts as the execution engine (Section 3.3). Finally, we integrate these components to illustrate the system's end-to-end workflow (Section 3.4).

3.1. Overview

The overall architecture of CurriculumPT is shown in Figure 1. At its core, the framework is designed to simulate the cognitive process of human experts, who master new skills by progressing from simple cases to complex problems. This paradigm is implemented through three tightly integrated components: curriculum scheduler, multi-agent system, and experience knowledge base. The curriculum scheduler organizes CVE-based tasks into a sequence of gradually increasing difficulty. The multi-agent system, composed of specialized LLM agents, then collaborates to execute these tasks. Throughout execution, actionable insights, including successful strategies, effective toolchains, and decision rationales, are continuously extracted and stored in the EKB. As the system advances through the curriculum, it retrieves and reuses this accumulated knowledge to address increasingly complex penetration scenarios. Together, these components form a closed-loop learning cycle that enables agents to continually refine their capabilities without model fine-tuning.

3.2. Curriculum-Guided Learning and Experience Accumulation

At the core of CurriculumPT lies a synergistic mechanism that combines a structured curriculum with an experience-driven reasoning loop. This section outlines the key components of the learning process: the design of the curriculum, the representation and application of experience, and the adaptive control of learning progression.

3.2.1. Task Difficulty-Based Curriculum Design

The construction of the curriculum begins with the classification and sequencing of penetration testing tasks (primarily based on CVE instances from Vulhub) according to their difficulty.

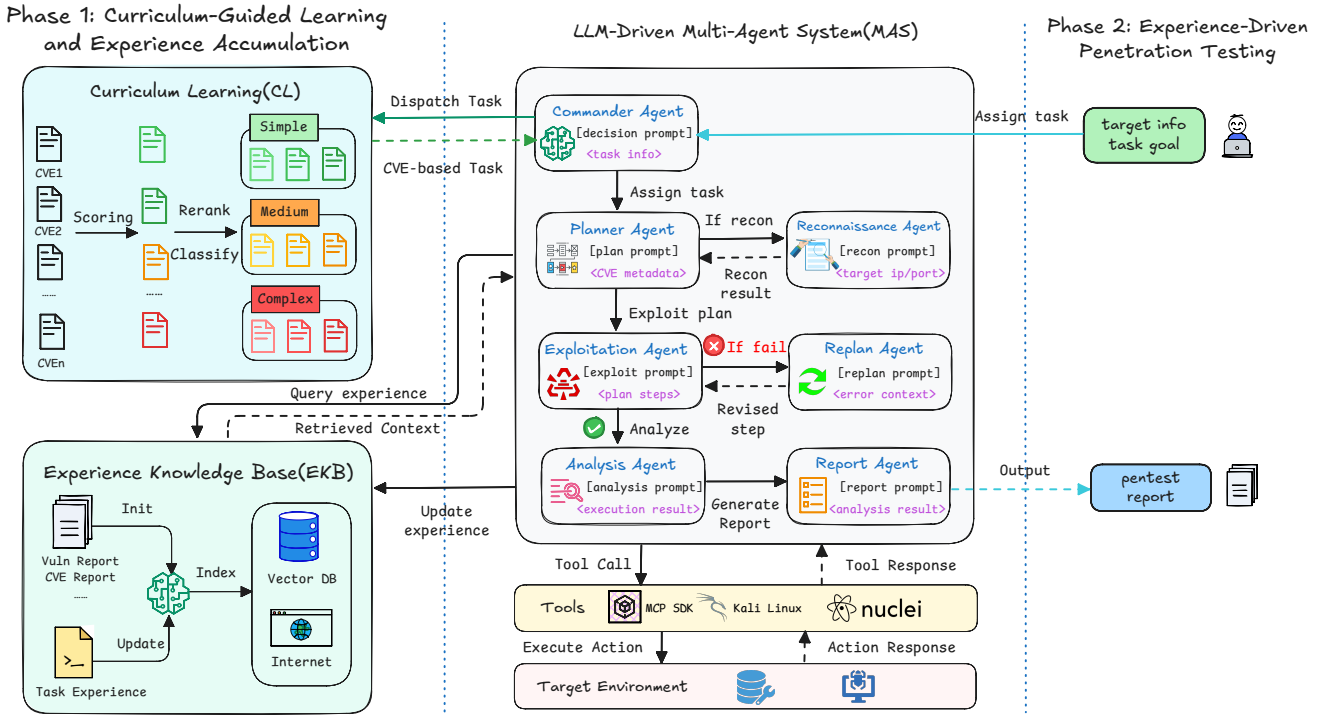

Figure 1. The architecture of the CurriculumPT framework.

Difficulty Metrics. We comprehensively assessed task difficulty based on the following factors: (1) AC—Attack Complexity: Derived from the Common Vulnerability Scoring System (CVSS) [37], this metric reflects the stringency of conditions required to exploit the vulnerability. A high value indicates that successful exploitation depends on specific environmental requirements or precise timing, thus increasing difficulty. (2) UI—User Interaction: Also based on CVSS, this measures whether successful exploitation requires victim user participation (e.g., clicking a malicious link or opening a file). Vulnerabilities marked as `Required` limit automation and increase difficulty. (3) PR—Privileges Required: Indicates the level of access an attacker must possess prior to exploiting the vulnerability. Higher privilege requirements (e.g., `Low` or `High`) imply additional prerequisite actions, such as local access or privilege escalation, making exploitation more complex. (4) ES—Exploitation Steps: Represents the number of distinct procedural steps needed to complete exploitation. Multi-stage exploits involving environment setup, payload crafting, chained execution, or post-exploitation steps are considered more difficult. ES is estimated using LLM-assisted analysis of CVE reproduction guides and public proof-of-concept (PoC) resources, such as scripts, technical blogs, or repositories that demonstrate real-world exploitation of reported vulnerabilities [38,39].

Curriculum Levels. To enable structured skill progression, all CVE-based penetration tasks are automatically categorized into three curriculum levels: Simple, Medium, and Complex. Task assignment is based on a unified difficulty scoring metric derived from four normalized and quantifiable indicators: AC, UI, PR, and ES. Each indicator is scaled to the range $[0, 1]$, and the overall difficulty score D is computed as a weighted linear combination:

$$D = \lambda_1 \cdot \mathrm{AC} + \lambda_2 \cdot \mathrm{UI} + \lambda_3 \cdot \mathrm{PR} + \lambda_4 \cdot \mathrm{ES} \quad (1)$$

where $\lambda_1 = 0.3$, $\lambda_2 = 0.2$, $\lambda_3 = 0.2$, and $\lambda_4 = 0.3$ denote the weights selected to reflect the contribution of each factor to the difficulty of exploitation. These values were determined

based on a qualitative analysis of sample CVEs, prioritizing AC and ES as they most directly influence the reasoning and action sequence required of the agent.

Based on the computed difficulty score, each vulnerability task is automatically assigned to one of three curriculum levels, reflecting different levels of exploitation complexity and reasoning demands. As shown in Table 1, tasks at Simple level typically involve low-complexity, single-step exploits using publicly available PoC scripts, with no need for user interaction or elevated privileges. Medium tasks require moderate effort, such as adapting PoCs or performing low-privilege, multi-step exploits. Complex tasks are the most challenging, often involving multi-stage exploit chains, advanced reasoning, and privilege escalation.

Table 1. Curriculum task level definition based on difficulty score.

Level	Difficulty Score Range	Task Characteristics
L_1—Simple	$0.0 \leq \text{score} < 0.4$	Low attack complexity (AC = Low) No user interaction (UI = None) No or low privileges required (PR = None/Low) Single-step exploitation (ES $\leq$ 2 steps) Public PoCs directly usable Examples: Default credentials, simple command injection, basic RCE
L_2—Medium	$0.4 \leq \text{score} < 0.7$	Moderate attack complexity (AC = Medium) May involve limited user interaction (UI = Required) Low privileges required (PR = Low) Multi-step exploitation (ES = 3–5 steps) PoCs require adaptation or chaining with reconnaissance Examples: File upload + WebShell, SQL injection with login bypass
L_3—Complex	$0.7 \leq \text{score} \leq 1.0$	High attack complexity (AC = High) User interaction often required (UI = Required) High privileges required or escalation needed (PR = High) Multi-stage exploitation with dependencies (ES $\geq$ 6 steps) No complete public PoCs; requires synthesis or chaining of techniques Examples: Logic chain exploits, RCE + privilege escalation, service-specific deserialization

3.2.2. Experience Representation

In this framework, "experience" refers to the structured recording of task execution processes, including successful strategies, useful intermediate results, and insights derived from both successes and failures. The Report Agent is responsible for generating and storing this experience. After each task attempt, regardless of success or failure, this agent analyzes the entire interaction process to extract valuable knowledge. It then organizes this information into several predefined, structured formats before storing them as new entries in the EKB. This captured knowledge includes comprehensive structured exploitation cases, saved as json objects detailing everything from the CVE ID to the exact command sequences used; practical problem–solution pairs that document specific errors and their resolutions; an atomic operation skills library containing efficient, context-specific commands for various services or protocols; and a high-quality prompt template library with validated prompts for recurring subtasks. By treating failed attempts and their causal analyzes as equally valuable, this process ensures that the EKB becomes a robust repository of actionable experiential knowledge.

3.2.3. Experience-Driven Reasoning

The critical process of experience transfer is initiated when the Planner Agent receives a new task. To assist with the current task, the system first retrieves relevant historical experiences from the EKB using a mechanism based on semantic similarity. It encodes the current task's description, including CVE characteristics, target service information, and expected vulnerability type, into a query vector, which is then compared against the vector representations of stored experience entries to return those with the highest similarity scores. The retrieved knowledge is then applied in multiple ways. The primary method is for the Planner Agent to leverage this experience for enhanced plan generation. It synthesizes the retrieved insights to construct more effective strategies, select optimal tools, and generate precise commands. These detailed plans and commands are then passed to the appropriate agents, such as the Exploitation Agent, for execution. For example, an agent may be guided by a prompt such as "You are trying to exploit [CVE-202X-YYYY]. According to the experience base, a similar vulnerability was successfully exploited using the following steps: [Step A, Command A]; [Step B, Command B;]. . . ". Furthermore, the retrieved experience is used to guide tool selection and parameter optimization based on successful past configurations and to assist the Planner Agent in high-level strategy and task decomposition by referencing how similar complex cases were previously resolved.

3.2.4. Adaptive Curriculum Pacing

The effective progression and adaptation of the system through the curriculum are essential to the success of our proposed CurriculumPT framework. To govern this process, we introduce a mechanism that controls both the advancement schedule and dynamic adjustments based on observed performance. The pacing function is managed by the Commander Agent, which monitors the system's overall performance on tasks at the current difficulty level using metrics like average success rate and number of attempts. When the system reaches a predefined proficiency threshold, such as an 80% exploit success rate on level N tasks, it begins to introduce tasks from the next difficulty level, N + 1. In addition, an adaptive mechanism addresses performance bottlenecks by injecting auxiliary tasks with slightly lower difficulty or modifying experience retrieval strategies to emphasize failed cases.

3.3. Curriculum-Guided Multi-Agent System

To support curriculum-guided penetration testing and structured experience reuse, we design a role-specialized multi-agent system (MAS) tailored for staged learning, performance monitoring, and adaptive decision making. In contrast to conventional MAS architectures, our design features tightly coupled task decomposition, curriculum progression control, and failure-aware replanning, collectively facilitating the acquisition of generalized exploitation skills. While this fine-grained specialization introduces more agents than some frameworks, it is a deliberate design choice. By isolating critical functions, such as learning control (Commander), failure analysis (Replan), and knowledge curation (Report), we prevent cognitive overload on a single planning agent and enable more robust, modular control over the entire learning loop. At the core of each agent lies a general-purpose LLM (e.g., GPT-4o [40]), which performs reasoning, plan generation, tool instruction translation, and output interpretation. The behavior of each agent is governed via carefully designed prompt templates under a unified coordination paradigm.

To enable these LLM agents, particularly the Reconnaissance Agent and the Exploitation Agent, to interact with actual penetration testing tools, we implemented a tool integration layer. A core feature of this layer is its use of an encapsulation mechanism of the Model Context Protocol (MCP) [41]. This mechanism abstracts a suite of commonly used penetration

testing tools, such as Nmap [42], Metasploit Framework [43], SQLMap [44], and Nikto [45] into standardized function calls. Such encapsulation not only provides a unified API for the designated agents but also simplifies the interaction logic between the LLM agents and diverse tools. The tool integration layer is responsible for accurately translating high-level instructions issued by the agents (under the guidance of their LLM core) into specific tool command line instructions, remotely invoking and securely executing these commands via the MCP architecture, and, finally, parsing the raw output returned by the tools into structured information that is easy for the LLM to understand and process further.

Building upon this foundation, each agent within the MAS is assigned specific roles and responsibilities:

- Commander Agent. Oversees task scheduling and curriculum pacing. It receives either a curriculum-level CVE task from the curriculum learning module (Phase 1) or a user-defined objective (Phase 2). Based on performance metrics (ESR), it dynamically adjusts curriculum progression or triggers adaptive auxiliary task injection.
- Planner Agent. Responsible for overall task planning. It receives metadata and reconnaissance results, queries the EKB, and synthesizes multi-step exploitation strategies. It decomposes complex procedures and coordinates execution with other agents.
- Reconnaissance Agent. Conducts information gathering on target hosts. Using tools like Nmap [42] and Nikto [45], it performs service enumeration, port scanning, directory probing, and returns structured system profiles to the Planner Agent.
- Exploitation Agent. Executes planned steps using toolkits (e.g., Metasploit [43], SQLMap [44]). For each step, it performs command execution and sends structured feedback (including success/failure indicators) to the Analysis Agent.
- Replan Agent. Triggered on failure, this agent analyzes contextual errors, tool output, and failed plans. It queries the EKB for related fix patterns and generates revised candidate steps to continue execution without restarting the full task.
- Analysis Agent. Evaluates execution outcomes. It determines whether subgoals were achieved and computes performance metrics (e.g., step latency, success ratio, number of replans). Results are returned to both the Commander Agent for learning control, and to the Report Agent for final documentation.
- Report Agent. Aggregates and abstracts complete execution traces, including success paths, critical parameters, encountered failures, and effective strategies. It updates the EKB with distilled experience entries in structured formats for future reuse.

3.4. Workflow

The workflow of the CurriculumPT framework is designed to enable progressive learning and effective generalization in automated penetration testing. It consists of two tightly coupled phases: (1) Curriculum-Guided Learning and Experience Accumulation, and (2) Experience-Driven Penetration Testing. These phases are not independent; instead, they form a closed learning cycle in which knowledge acquired during Phase 1 directly supports decision making in Phase 2, while execution results from Phase 2 further refining the knowledge base. This two-phase design enables CurriculumPT to not only learn incrementally but also adapt and apply its knowledge to novel, real-world scenarios, ensuring both robustness and continual improvement.

Phase 1: Curriculum-Guided Learning and Experience Accumulation. The goal of this phase is to systematically bootstrap the MAS and build foundational penetration capabilities through staged, difficulty-aware learning, as detailed in Algorithm 1. The process begins with the Commander Agent, which coordinates the entire workflow by selecting a candidate task from the current curriculum level. The task is then forwarded to the Planner Agent, which queries the EKB for relevant prior knowledge, and, if necessary,

invokes the Reconnaissance Agent to gather target information before generating a detailed exploitation plan. The plan is subsequently executed by the Exploitation Agent. If any step fails, a ReAct-style loop is triggered: the Replan Agent analyzes the failure context, queries the EKB for similar error resolutions, and generates revised steps to continue the attempt. This execution–replanning cycle continues until the task is successfully completed or a maximum number of attempts is reached. Upon completion, the Analysis Agent evaluates the outcome and extracts key performance metrics, which are sent to the Commander Agent to track progress. Concurrently, the Report Agent summarizes the entire execution trace into a structured experience entry and stores it in the EKB. Finally, the Commander Agent uses the aggregated performance metrics to check if the learning objectives at the current difficulty level have been achieved, and accordingly decides whether to advance to the next level or continue training.

Algorithm 1 Curriculum-Guided Learning and Experience Accumulation.

Input: Curriculum Levels $\mathcal{L} = \{L_1, \ldots, L_n\}$; Task Pool $\mathcal{T}$ with CVE metadata; Max Replanning Iterations $N_{\max}$; Completion Thresholds θ
Output: Updated Experience Knowledge Base (EKB)
Initialize EKB $\mathcal{E} \leftarrow \emptyset$
Initialize Progress Tracker $\mathcal{P} \leftarrow \emptyset$
for each level $L_i \in \mathcal{L}$ **do**
  while NotCompleted$(\mathcal{P}, L_i, \theta)$ **do**
    $t \leftarrow$ CommanderAgent.SelectTask$(\mathcal{T}_{L_i}, \mathcal{P})$
    $exp \leftarrow$ QueryEKB$(\mathcal{E}, t)$
    $plan \leftarrow$ PlannerAgent.Plan(t, exp)
    if PlannerAgent.RequiresRecon$(plan)$ **then**
      $info \leftarrow$ ReconnaissanceAgent(t)
      $plan \leftarrow$ PlannerAgent.ReplanWithInfo$(plan, info)$
    end if
    $success \leftarrow$ **false**, $attempt \leftarrow 0$
    while $\neg success$ **and** $attempt < N_{\max}$ **do**
      $result \leftarrow$ ExploitationAgent.Execute$(plan)$
      if IsSuccess$(result)$ **then**
        $success \leftarrow$ **true**
      else
        $plan \leftarrow$ ReplanAgent$(result, plan, \mathcal{E})$
        $attempt \leftarrow attempt + 1$
      end if
    end while
    $log \leftarrow$ LogExecution$(result, plan)$
    $metrics \leftarrow$ AnalysisAgent.Evaluate(log)
    CommanderAgent.UpdateProgress$(\mathcal{P}, t, metrics)$
    $entry \leftarrow$ ReportAgent.Summarize(log)
    $\mathcal{E} \leftarrow \mathcal{E} \cup \{entry\}$
  end while
end for
return $\mathcal{E}$

Phase 2: Experience-Driven Penetration Testing. The second phase assesses the framework's generalization capability by applying the knowledge accumulated in Phase 1 to solve real-world security challenges. Transitioning from curriculum-driven learning to practical application, this phase is initiated by a user-defined task, which can be an unstructured, high-level objective (e.g., "try to read sensitive file/etc/passwd"). Upon receiving this goal, the Commander Agent orchestrates the agent pipeline. A key feature of this phase is the experience-driven planning strategy employed by the Planner Agent. It proactively queries the EKB, and, upon finding a highly relevant precedent, may generate

a direct exploitation plan, potentially bypassing unnecessary reconnaissance steps. The framework then leverages the same execution and replanning loop from Phase 1 to attempt the task. After the attempt, the Analysis Agent assesses the final outcome against the user's objective and generates performance metrics. Importantly, the learning loop remains active. The Report Agent processes the entire execution trace, including successful strategies and failure recovery paths, and transforms them into new structured entries. This ensures that the experience knowledge base (EKB) is continuously enriched with diverse, real-world scenarios.

4. Evaluation

In this section, we validate the effectiveness of CurriculumPT.

RQ1 (Overall Performance): How effective is the CurriculumPT framework overall?

RQ2 (Comparison Analysis): What are the specific advantages of CurriculumPT over existing LLM-based penetration testing methods, particularly regarding (1) progressive task learning via curriculum guidance, (2) modular agent coordination for task decomposition, and (3) experience-driven adaptation and generalization through EKB reuse?

RQ3 (Ablation Study): What is the contribution of each core component in the CurriculumPT framework?

4.1. Experimental Setting

(1) Benchmark Dataset. All experiments in this study were conducted using the Vulhub vulnerability reproduction platform. Vulhub [46] is a widely used open-source project built on Docker and Docker Compose, offering a broad collection of real-world vulnerability environments. It provides standardized and reproducible conditions for penetration testing and security research. To construct a structured evaluation dataset, we selected a representative subset of CVE from Vulhub. The selection criteria emphasized both the diversity of vulnerability types (e.g., remote code execution, SQL injection, file upload, command injection, directory traversal) and variation in exploitation complexity. Each CVE instance was automatically classified into one of three curriculum difficulty levels—Simple, Medium, or Complex, according to a unified difficulty scoring metric based on four normalized indicators: Attack Complexity (AC), User Interaction (UI), Privileges Required (PR), and Exploitation Steps (ES). This classification reflects both procedural and cognitive challenges in exploitation and underpins the task sequencing used in our curriculum learning strategy. To evaluate generalization, a subset of CVEs was excluded from training and reserved solely for downstream testing. While the experiments are based on general-purpose CVE environments, the selected vulnerabilities are representative of those frequently encountered in intelligent transportation systems (ITS). These include traffic signal controllers, roadside communication gateways, and vehicular cloud services, which often run on web-based management platforms or embedded Linux devices. As such, the benchmark scenarios are highly relevant for assessing cybersecurity risks and resilience in ITS environments.

(2) Metrics. To conduct a comprehensive and multi-dimensional evaluation of different system configurations, this study adopts a set of key quantitative metrics.

- Exploitation Success Rate (ESR). This is the primary metric for assessing the core effectiveness of the system. It reflects the ability to successfully reproduce CVEs and achieve defined objectives (e.g., obtaining shell access or reading sensitive files).

$$\mathrm{ESR} = \frac{N_{\mathrm{success}}}{N_{\mathrm{total}}} \tag{2}$$

where N_{success} denotes the number of CVE exploitation tasks that successfully achieve the target objective, and N_{total} is the total number of exploitation attempts.

- Average Steps per Task (AST). AST measures the average number of reasoning–execution iterations required to complete a successful task. Each step typically involves a planning decision, tool invocation, or replanning operation. This metric reflects both interaction depth and coordination overhead.

$$\text{AST} = \frac{S_{\text{total}}}{N_{\text{success}}} \tag{3}$$

where S_{total} is the total number of planner–agent interactions across all successful tasks.

- Average Time to Exploit (ATE). ATE quantifies the average time required to successfully exploit a vulnerability, from initiation to completion. It serves as an indicator of overall system execution efficiency.

$$\text{ATE} = \frac{T_{\text{total}}}{N_{\text{success}}} \tag{4}$$

where T_{total} is the total time consumed across all successful exploits.

- Average Token Usage (ATU). This metric measures the average number of tokens (including both prompt and completion) consumed per task, providing a fine-grained estimation of reasoning overhead and cost-efficiency.

$$\text{ATU} = \frac{T_{\text{tokens}}}{N_{\text{task}}} \tag{5}$$

where T_{tokens} is the total number of tokens used in LLM invocations, and N_{task} is the number of penetration testing tasks. Lower ATU values indicate more efficient reasoning.

- Experience Knowledge Base Hit Rate (EHR). EHR evaluates the effectiveness of the EKB in supporting decision making during task execution. It measures how frequently retrieved experience is successfully applied to assist in solving new tasks.

$$\text{EHR} = \frac{N_{\text{hit}}}{N_{\text{retrieval}}} \tag{6}$$

where N_{hit} denotes the number of successful applications of retrieved experience, and $N_{\text{retrieval}}$ represents the total number of retrieval attempts.

(3) Implemental Details. All experiments were conducted on a laptop equipped with an Intel Core i7-14650HX processor (2.20 GHz) and 32 GB RAM. The vulnerable environments were deployed using Docker containers based on official Vulhub images, running within an Ubuntu virtual machine. The attacker-side was hosted on a separate Kali Linux virtual machine, which served as the orchestrator via a customized MCP interface. Both virtual machines were connected through NAT within a local network, simulating realistic internal conditions while ensuring isolation. All LLM agents in CurriculumPT were powered by GPT-4o-mini, accessed through OpenAI API, and used for planning, reasoning, code generation, and tool interaction.

4.2. Performance Evaluation

The results in Table 2 illustrate the effectiveness and efficiency of the CurriculumPT framework across progressive curriculum stages. As task difficulty increases from Level 1 (Simple) to Level 3 (Complex), the ESR decreases from 95.3% to 60.0%, reflecting the growing challenge associated with more complex vulnerability tasks. Correspondingly, the ATE increases from 110 to 370 s, and the ATU rises from 2.3M to 5.6M tokens, indicating

higher reasoning and interaction overhead in more complex scenarios. Notably, the EHR improves significantly with task complexity, rising from 67.9% at Level 1 to 81.7% at Level 3. This trend suggests that experiential reuse becomes increasingly valuable and effective as challenges intensify. Performance on the hold-out set, composed of mixed-difficulty CVEs not seen during training, achieves a competitive ESR of 66.7%, and maintains a high EHR of 80.0%, demonstrating strong generalization of the learned knowledge.

Table 2. Performance of CurriculumPT across different curriculum levels and on a hold-out set.

Curriculum Level (No. of CVEs)	ESR (%)	ATE (s)	EHR (%)	ATU (M Tokens)
Level 1: Simple (N = 15 CVEs)	95.3 ± 4.7	110 ± 15	67.9 (91/134)	2.3 ± 0.4
Level 2: Medium (N = 20 CVEs)	75.0 ± 7.1	180 ± 25	77.9 (152/195)	3.7 ± 0.6
Level 3: Complex (N = 15 CVEs)	60.0 ± 8.1	370 ± 85	81.7 (192/235)	5.6 ± 0.9
Hold-out Set (N = 15 CVEs, mixed difficulty)	66.7 ± 7.8	340 ± 70	80.0 (120/150)	5.1 ± 0.8

Answering RQ1

CurriculumPT demonstrates stable and strong performance in automated penetration testing, maintaining high success rates across varying task complexities. It achieves an exploitation success rate (ESR) of 66.7% on previously unseen CVEs, validating its adaptability and knowledge generalization capabilities.

4.3. Comparison Analysis

To highlight the specific advantages of CurriculumPT over existing LLM-based penetration testing frameworks, we performed a comparative analysis involving three representative baselines: AutoPT [15], VulnBot [16], and PentestAgent [18]. The comparison focuses on both framework characteristics and empirical performance across 15 standardized penetration testing scenarios. Each framework was implemented according to its original design specifications and executed under identical conditions, including the same underlying LLM (GPT-4o-mini) and consistent environment settings, to ensure fair comparison.

Table 3 presents a comparative analysis between CurriculumPT and three representative LLM-based penetration testing frameworks: AutoPT, VulnBot, and PentestAgent, focusing on both architectural features and empirical performance. Among the four, CurriculumPT is the only framework that integrates three targeted architectural features. It uniquely supports curriculum learning, enabling agents to progressively acquire skills aligned with increasing task complexity. While VulnBot and PentestAgent adopt modular agent architectures, they lack curriculum guidance and full EKB reuse, which limits their adaptability and generalization. Empirically, CurriculumPT outperforms all baselines across every performance metric. It achieves the highest ESR of 60.0%, demonstrating superior task effectiveness. Additionally, it completes tasks with the shortest ATE of 390 s, the lowest ATU of 3.5 M, and the fewest AST of 5.1. These results indicate that CurriculumPT's architectural innovations, particularly curriculum learning and experience reuse, contribute directly to more efficient and effective penetration testing. Overall, these results demonstrate that CurriculumPT not only introduces meaningful design innovations but also translates them into measurable advantages in real-world LLM-driven penetration testing.

Table 3. Comparison of CurriculumPT with representative LLM-based penetration-testing frameworks on hold-out set (N = 15).

Method	Key Design Features			Performance			
	CL	MAS	EKB/RAG	ESR (%)	ATE (s)	ATU (M Tokens)	AST
CurriculumPT (ours)	✓	✓	✓	**60.0**	**390** ± 80	**3.5** ± 0.7	**5.1**
AutoPT [15]	×	×	×	24.0	620 ± 110	6.1 ± 1.3	7.4
VulnBot [16]	×	✓	△	36.0	540 ± 95	5.2 ± 1.0	6.3
PentestAgent [18]	×	✓	✓	42.0	490 ± 90	4.7 ± 0.9	8.2

✓ = feature present; × = feature absent; △ = partially implemented. Bold values indicate the best performance in the corresponding column.

Answering RQ2

CurriculumPT demonstrates clear advantages over existing LLM-based penetration testing frameworks by uniquely integrating curriculum learning, modular agent coordination, and reuse of experience knowledge, resulting in superior task success rates, efficiency, and generalization.

4.4. Ablation Study

To evaluate the contributions of core components in the CurriculumPT framework across varying difficulty levels, we perform a comprehensive ablation study on a stratified CVE test set (N = 30), consisting of 10 Simple, 10 Medium, and 10 Complex vulnerability scenarios. All CVEs in this set are excluded from the curriculum training phase to avoid data leakage and ensure fair evaluation. All experiments are conducted under consistent system configurations using the same GPT-4o-mini model (128 k context window, temperature = 0) and identical runtime environments. Each CVE is tested once per system variant under fixed resource constraints. We examine four system variants to assess the contributions of curriculum learning and experience knowledge reuse. The Full configuration includes both CL and the EKB, representing the complete framework. In the No EKB setting, curriculum scheduling is retained, but experiential reuse is limited to the LLM's internal context window, with no access to external memory. The No CL configuration disables curriculum-guided task progression, presenting tasks in a random order while retaining the EKB. Finally, the No CL + No EKB variant removes both staged learning and long-term experience storage, serving as a minimal baseline for comparison.

As shown in Table 4, the full configuration of CurriculumPT consistently achieves superior performance across all difficulty levels. It reaches the highest ESR of 100% on simple tasks and maintains 60.0% on complex ones, while also exhibiting the lowest ATE, token consumption (ATU), and reasoning steps (AST). In contrast, disabling the experience knowledge base (No EKB) significantly impairs performance, particularly on complex tasks. The system's reliance on the limited short-term context of the LLM results in a drop in ESR to 43.3%, accompanied by substantial increases in both ATE and ATU. Similarly, removing curriculum learning (No CL) disrupts the structured progression of task complexity. Although the EKB remains available, the absence of curriculum guidance leads to reduced performance due to the lack of systematic skill accumulation. The most pronounced degradation is observed in the No CL + No EKB configuration, where both structured learning and long-term memory are eliminated. This variant exhibits the lowest ESR (33.3%), the highest ATE (450 s), and the greatest token consumption and reasoning overhead, especially on medium and complex CVEs. These results collectively demonstrate that curriculum guidance and experience reuse contribute complementary and indispensable benefits.

Table 4. Ablation study of CurriculumPT components across different difficulty levels.

Variants	ESR (%)			ATE (s)			ATU (M Tokens)			AST		
	Simple	Medium	Complex	Simple	Medium	Complex	Simple	Medium	Complex	Simple	Medium	Complex
Full	**100**	**75.0**	**60.0**	**110**	**180**	**370**	**2.3**	**3.7**	**5.6**	**3.2**	**4.0**	**5.5**
No EKB	86.7	60.0	43.3	140	210	400	2.8	4.1	5.9	3.5	4.4	5.8
No CL	80.0	58.3	40.0	130	200	390	2.7	4.0	5.6	3.4	4.3	5.7
No CL + No EKB	73.3	50.0	33.3	150	230	450	3.2	4.4	6.2	3.7	4.6	6.0

Answering RQ3

The results show that curriculum learning effectively guides LLMs to acquire and generalize penetration testing skills, while the EKB enhances efficiency through experience reuse. Together, they significantly boost success rates and reduce execution overhead.

5. Discussion

This work on CurriculumPT, which introduces a curriculum learning paradigm to multi-agent penetration testing systems, has demonstrated significant potential. This section further contextualizes our core contributions, discusses the challenges of real-world deployment, and candidly addresses the limitations of our current study.

5.1. Core Contributions

The central contribution of CurriculumPT lies in its reformulation of automated penetration testing from a planning problem to a learning problem. Unlike conventional methods that depend on static knowledge augmentation (e.g., RAG [47]) or predefined control flows (e.g., FSM [15]), our curriculum-guided method posits that generalization arises from progressive skill acquisition, analogous to human learning. This learning-centric perspective shifts the objective from merely solving individual tasks to developing an agent's ability to internalize and transfer experiential knowledge. By demonstrating that an LLM-based system can "grow" through practice without explicit parameter fine-tuning, CurriculumPT provides a proof-of-concept for a new direction in cybersecurity AI: building autonomous agents that learn how to learn, enabling more adaptive and resilient responses to emerging threats.

5.2. Challenges for Real-World Performance

Although extensive experiments have validated the effectiveness of CurriculumPT in controlled environments such as Vulhub, deploying it in complex real-world networks presents several challenges. First, the dynamism and heterogeneity of real-world systems far exceed those of standardized testing environments. Target environments may have deployed advanced intrusion detection/prevention systems (IDS/IPS), web application firewalls (WAFs), and honeypots [48], among other active defense measures, all of which can interfere with and obstruct the reconnaissance, analysis, and exploitation behaviors of automated tools. CurriculumPT must improve its environmental awareness, dynamic adaptability, and the ability to generate stealthy strategies. Second, the "window of opportunity" for vulnerability exploitation can be extremely brief in the real world, requiring high responsiveness and execution efficiency. The current reliance on large LLM API calls may introduce non-negligible latency, potentially limiting operational timeliness. Moreover, the exploitation of "zero-day" or "one-day" vulnerabilities typically lacks structured, publicly available knowledge. This places higher demands on the system's generalization and creative exploit generation capabilities. Whether the "meta-experience" accumulated through curriculum learning can be effectively transferred to such scenarios remains an open and critical question.

5.3. Limitations

Despite its promising results, this study has several limitations. First, the current methods of experience representation in the knowledge base, such as structured exploitation cases and reusable prompt templates, may not fully capture the nuanced and context-dependent nature of real-world penetration testing. Addressing this limitation will require more abstract and flexible reasoning mechanisms. Second, the performance of the framework is inherently limited by the capabilities of the underlying language model, including limited context windows, occasional reasoning instability, potential for hallucinated outputs [49]. In addition, the use of LLM APIs introduces practical constraints such as high computational costs and rate limits. Third, although the evaluation was based on real-world vulnerabilities, the experiments were conducted in sandboxed environments using Vulhub. This setup, while reproducible and controlled, does not reflect the full complexity of enterprise-grade systems that often deploy live defenses such as intrusion detection systems, rate-limiting and authentication mechanisms. Furthermore, our selection of vulnerability scenarios may introduce unintentional bias toward vulnerabilities with publicly available proof-of-concept exploits, which are more likely to be solvable by automated agents. Future work will explore broader and more diverse vulnerability sets and test the framework under more realistic, constrained environments to better assess its robustness and practical applicability.

5.4. Ethical Considerations

Given the dual-use nature of penetration testing tools, CurriculumPT has been developed with explicit ethical safeguards to prevent misuse and ensure responsible deployment. All experiments presented in this work were performed within isolated, sandboxed environments to eliminate any risk to external systems. For real-world applications, the framework requires explicit consent from the system owner and a strictly defined operational scope. To further reduce the potential for unintended system impact, CurriculumPT incorporates human-in-the-loop oversight, execution throttling, and automatic termination in response to anomalous behavior. Comprehensive logging of all actions enables full traceability and auditability. While the system is capable of high autonomy, its use must remain fully compliant with legal and ethical standards. CurriculumPT is not intended for offensive or unauthorized activities. Responsible use of the framework is grounded in established ethical principles, including informed consent, transparency, and the minimization of harm.

6. Conclusions

This paper presented CurriculumPT, a novel automated penetration testing framework that pioneers a curriculum-guided learning approach. By simulating how human experts master skills through progressively complex tasks, CurriculumPT enables LLM agents to systematically accumulate and transfer experience, significantly enhancing their success rate and generalization on complex CVEs without costly model fine-tuning. The experimental results demonstrate that CurriculumPT significantly outperforms baselines that lack its learning-centric, multi-agent architecture. Furthermore, ablation studies confirmed that each core component—curriculum learning, the experience knowledge base, and specialized agent design—is indispensable for achieving the framework's high performance and efficiency. Notably, through curriculum learning, the system's experience utilization efficiency improved with increasing task complexity, and it displayed promising generalization potential on a hold-out set. While this study has limitations, including the scope of the dataset, the degree of automation in curriculum design, and the depth of experience representation, it provides a valuable exploration into enabling LLMs to achieve a "learning to learn" capability within complex professional domains. CurriculumPT also lays the groundwork for building more intelligent, adaptive, and autonomous cybersecurity systems.

Looking ahead, the CurriculumPT framework and its core concepts open several promising research directions for advancing automated penetration testing. As a first step toward real-world deployment, we plan to enhance the learning process through dynamic curriculum generation and more robust reasoning models to support knowledge transfer across heterogeneous systems. We also aim to extend CurriculumPT by integrating real-time adaptation to dynamic targets and network conditions, enabling the framework to operate effectively in evolving environments. To ensure safety and operational oversight, we will incorporate human-in-the-loop mechanisms for supervising high-risk actions and validating system decisions. Additionally, we are exploring multimodal information processing and human–AI collaborative paradigms to improve task coordination and explainability. Finally, addressing ethical implications and defensive countermeasures remains essential to the responsible application and governance of this technology.

Author Contributions: Conceptualization, X.W.; methodology, X.W. and Y.T.; validation, X.W.; formal analysis, X.W., Y.C. and J.J.; investigation, X.W., P.Y. and Y.T.; resources, X.W., X.C. and S.L.; data curation, X.W., S.L. and J.J.; writing—original draft preparation, X.W.; writing—review and editing, X.W. and Y.T.; visualization, X.W. and Y.T.; supervision, J.L. and W.N.; project administration, J.L. and W.N.; funding acquisition, J.L. and W.N. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Central Funds Guiding the Local Science and Technology Development under Grant No. 236Z0806G, the Fundamental Research Funds for the Central Universities under Grant No. 2023JBMC055, and the National Natural Science Foundation of China under Grant No. 62372021, China.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in the study are included in the article, further inquiries can be directed to the corresponding author.

Conflicts of Interest: Author Shouyang Li was employed by the company China Railway Qinghai Tibet Group Co., Ltd. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Arkin, B.; Stender, S.; McGraw, G. Software penetration testing. *IEEE Secur. Priv.* **2005**, *3*, 84–87. [CrossRef] [CrossRef]
2. Bishop, M. About penetration testing. *IEEE Secur. Priv.* **2007**, *5*, 84–87. [CrossRef] [CrossRef]
3. Umrao, S.A.C.H.I.N.; Kaur, M.A.N.D.E.E.P.; Gupta, G.K. Vulnerability assessment and penetration testing. *Int. J. Comput. Commun. Technol.* **2012**, *3*, 71–74. [CrossRef]
4. Abu-Dabaseh, F.; Alshammari, E. Automated penetration testing: An overview. In Proceedings of the 4th International Conference on Natural Language Computing, Dubai, UAE, 28–29 April 2018; pp. 121–129.
5. Bilge, L.; Dumitraş, T. Before we knew it: An empirical study of zero-day attacks in the real world. In Proceedings of the 2012 ACM Conference on Computer and Communications Security, Raleigh, NC, USA, 16–18 October 2012; pp. 833–844. [CrossRef]
6. Wang, X.; Sun, K.; Batcheller, A.; Jajodia, S. Detecting "0-day" vulnerability: An empirical study of secret security patch in oss. In Proceedings of the 2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), Portland, OR, USA, 24–27 June 2019; pp. 485–492. [CrossRef]
7. Tudosi, A.D.; Graur, A.; Balan, D.G.; Potorac, A.D. Research on security weakness using penetration testing in a distributed firewall. *Sensors* **2023**, *23*, 2683. [CrossRef] [CrossRef]
8. Chowdhary, A.; Jha, K.; Zhao, M. Generative adversarial network (gan)-based autonomous penetration testing for web applications. *Sensors* **2023**, *23*, 8014. [CrossRef] [CrossRef]
9. Berenguer, A.; Morejón, A.; Tomás, D.; Mazón, J.N. Large language models to enhance the reusability of sensor data. *Sensors* **2024**, *24*, 347. [CrossRef] [CrossRef]

10. Fu, M.; Tantithamthavorn, C. K.; Nguyen, V.; Le, T. ChatGPT for vulnerability detection, classification, and repair: How far are we? In Proceedings of the 2023 30th Asia-Pacific Software Engineering Conference (APSEC), Seoul, Republic of Korea, 4–7 December 2023; pp. 632–636. [CrossRef]
11. Happe, A.; Cito, J. Getting pwn'd by ai: Penetration testing with large language models. In Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, San Francisco, CA, USA, 3–9 December 2023; pp. 2082–2086.
12. Deng, G.; Liu, Y.; Mayoral-Vilches, V.; Liu, P.; Li, Y.; Xu, Y.; Zhang, T.; Liu, Y.; Pinzger, M.; Rass, S. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24), Philadelphia, PA, USA, 14–16 August 2024; pp. 847–864.
13. Fırat, M.; Kuleli, S. What if GPT4 became autonomous: The Auto-GPT project and use cases. *J. Emerg. Comput. Technol.* **2023**, *3*, 1–6. [CrossRef]
14. Xu, J.; Stokes, J. W.; McDonald, G.; Bai, X.; Marshall, D.; Wang, S.; Swaminathan, A.; Li, Z. Autoattacker: A large language model guided system to implement automatic cyber-attacks. *arXiv* **2024**, arXiv:2403.01038. [CrossRef]
15. Wu, B.; Chen, G.; Chen, K.; Shang, X.; Han, J.; He, Y.; Zhang, W.; Yu, N. Autopt: How far are we from the end2end automated web penetration testing? *arXiv* **2024**, arXiv:2411.01236.
16. Kong, H.; Hu, D.; Ge, J.; Li, L.; Li, T.; Wu, B. VulnBot: Autonomous penetration testing for a multi-agent collaborative framework. *arXiv* **2025**, arXiv:2501.13411.
17. Gao, L.; Liu, Y.; Chen, H.; Liu, D.; Zhang, Y.; Sun, J. Exploring traffic simulation and cybersecurity strategies using large language models. In Proceedings of the 2025 IEEE Security and Privacy Workshops (SPW), San Francisco, CA, USA, 15 May 2025; pp. 346–351. [CrossRef]
18. Shen, X.; Wang, L.; Li, Z.; Chen, Y.; Zhao, W.; Sun, D.; Wang, J.; Ruan, W. PentestAgent: Incorporating llm agents to automated penetration testing. *arXiv* **2024**, arXiv:2411.05185. [CrossRef]
19. Faillon, M.-A.; Bout, B.; Francq, J.; Neal, C.; Boulahia-Cuppens, N.; Cuppens, F.; Yaich, R. How to better fit reinforcement Learning for pentesting: A new hierarchical approach. In Proceedings of the European Symposium on Research in Computer Security, Bydgoszcz, Poland, 16–20 September 2024; pp. 313–332. [CrossRef]
20. Li, Q.; Zhang, M.; Shen, Y.; Wang, R.; Hu, M.; Li, Y.; Hao, H. A hierarchical deep reinforcement learning model with expert prior knowledge for intelligent penetration testing. *Comput. Secur.* **2023**, *132*, 103358. [CrossRef] [CrossRef]
21. Chen, J.; Hu, S.; Zheng, H.; Xing, C.; Zhang, G. GAIL-PT: An intelligent penetration testing framework with generative adversarial imitation learning. *Comput. Secur.* **2023**, *126*, 103055. [CrossRef] [CrossRef]
22. Schwartz, J. Network Attack Simulator. 2020. Available online: https://github.com/Jjschwartz/NetworkAttackSimulator (accessed on 15 May 2025).
23. Standen, M.; Lucas, M.; Bowman, D.; Richer, T. J.; Kim, J.; Marriott, D. Cyborg: A gym for the development of autonomous cyber agents. *arXiv* **2021**, arXiv:2108.09118. [CrossRef]
24. Hazell, J. Large language models can be used to effectively scale spear phishing campaigns. *arXiv* **2023**, arXiv:2305.06972.
25. Hilario, E.; Azam, S.; Sundaram, J.; Imran Mohammed, K.; Shanmugam, B. Generative ai for pentesting: The good, the bad, the ugly. *Int. J. Inf. Secur.* **2024**, *23*, 2075–2097. [CrossRef] [CrossRef]
26. Wang, L.; Wang, J.; Jung, K.; Thiagarajan, K.; Wei, E.; Shen, X.; Chen, Y.; Li, Z. From sands to mansions: Enabling automatic full-life-cycle cyberattack construction with llm. *arXiv* **2024**, arXiv:2407.16928.
27. Muzsai, L.; Imolai, D.; Lukács, A. HackSynth: Llm agent and evaluation framework for autonomous penetration testing. *arXiv* **2024**, arXiv:2412.01778. [CrossRef]
28. Bianou, S.G.; Batogna, R.G. PENTEST-AI, an llm-powered multi-agents framework for penetration testing automation leveraging mitre attack. In Proceedings of the 2024 IEEE International Conference on Cyber Security and Resilience (CSR), London, UK, 2–4 September 2024; pp. 763–770. [CrossRef]
29. Udeshi, M.; Shao, M.; Xi, H.; Rani, N.; Milner, K.; Sai Charan Putrevu, V.; Shafique, M. D-CIPHER: Dynamic collaborative intelligent agents with planning and heterogeneous execution for enhanced reasoning in offensive security. *arXiv* **2025**, arXiv:2502.10931.
30. Nakatani, S. RapidPen: Fully automated ip-to-shell penetration testing with llm-based agents. *arXiv* **2025**, arXiv:2502.16730.
31. Bengio, Y.; Louradour, J.; Collobert, R.; Weston, J. Curriculum learning. In Proceedings of the 26th Annual International Conference on Machine Learning, Montreal, QC, Canada, 14–18 June 2009; pp. 41–48. [CrossRef]
32. Du, Q.; Kun, W.; Kuang, X.; Li, X.; Zhao, G. Automated software vulnerability detection via curriculum learning. In Proceedings of the 2023 IEEE International Conference on Multimedia and Expo (ICME), Brisbane, Australia, 10–14 July 2023; pp. 2855–2860. [CrossRef]
33. Shi, T.; Wu, Y.; Song, L.; Zhou, T.; Zhao, J. Efficient reinforcement finetuning via adaptive curriculum learning. *arXiv* **2025**, arXiv:2504.05520. [CrossRef]

34. Vu, D. A.; Nguyen, C.-D.; Wu, X.; Hoang, N.; Du, M.; Nguyen, T.; Luu, A. T. Curriculum demonstration selection for in-context learning. In Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing, Sicily, Italy, 31 March–4 April 2025; pp. 1004–1006. [CrossRef]
35. Ma, X.; Jiang, W.; Huang, H. Problem-Solving logic guided curriculum in-context learning for llms complex reasoning. *arXiv* **2025**, arXiv:2502.15401.
36. Zhang, E.; Yan, X.; Lin, W.; Zhang, T.; Lu, Q. Learning like humans: Advancing llm reasoning capabilities via adaptive difficulty curriculum learning and expert-guided self-reformulation. *arXiv* **2025**, arXiv:2505.08364. [CrossRef]
37. Mell, P.; Spring, J.; Dugal, D. *Measuring the Common Vulnerability Scoring System Base Score Equation*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2022.
38. OffSec. Exploit Database-Exploits, Shellcode, and Security Papers. Available online: https://www.exploit-db.com/ (accessed on 4 August 2025).
39. Wang, Y.; Wu, W.; Zhang, C.; Xing, X.; Gong, X.; Zou, W. From proof-of-concept to exploitable: (One step towards automatic exploitability assessment). *Cybersecurity* **2019**, *2*, 12. [CrossRef]
40. Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S. Gpt-4 technical report. *arXiv* **2023**, arXiv:2303.08774. [CrossRef]
41. Model Context Protocol. Introduction to Model Context Protocol (mcp). Available online: https://modelcontextprotocol.io/introduction (accessed on 15 May 2025).
42. Nmap. Nmap-The Network Mapper. Available online: https://nmap.org/ (accessed on 15 May 2025).
43. Rapid7. Metasploit Framework. Available online: https://www.metasploit.com/ (accessed on 15 May 2025).
44. Sqlmapproject. Sqlmap: Automatic SQL Injection and Database Takeover Tool. Available online: https://sqlmap.org/ (accessed on 15 May 2025).
45. Sullo. Nikto. Available online: https://github.com/sullo/nikto (accessed on 15 May 2025).
46. vulhub. Vulhub: Pre-Built Vulnerable Environments Based on Docker-Compose. Available online: https://vulhub.org/ (accessed on 15 May 2025).
47. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 9459–9474.
48. Kambow, N.; Passi, L.K. Honeypots: The need of network security. *Int. J. Comput. Sci. Inf. Technol.* **2014**, *5*, 6098–6101.
49. Ji, Z.; Lee, N.; Frieske, R.; Yu, T.; Su, D.; Xu, Y.; Fung, P. Survey of hallucination in natural language generation. *ACM Comput. Surv.* **2020**, *55*, 1–38. [CrossRef]